

DECREASE AND CONQUER:

* 3 TYPES OF DECREASE AND CONQUER:

1) decrease by constant

3) variable size decrease.

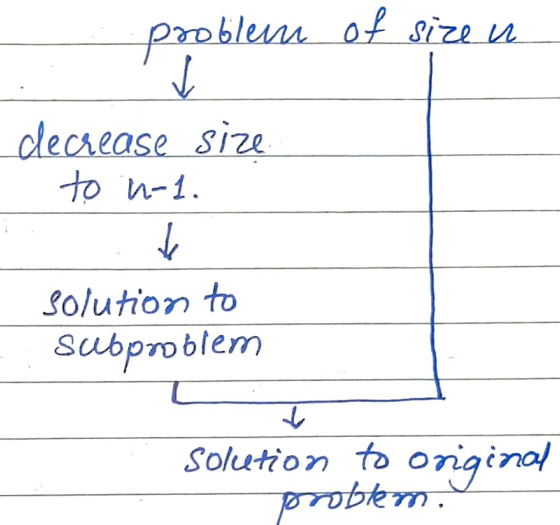
2) decrease by constant factor

1) DECREASE BY CONSTANT:

$$a^n = a^{n-1} \cdot a$$

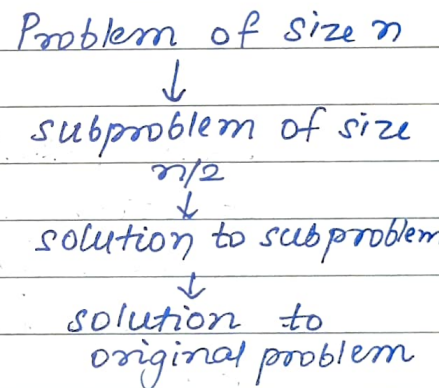
$$f(n) = a^n$$

$$f(n) = \begin{cases} f(n-1) \cdot a & \text{if } n > 0 \\ 1 & \text{if } n = 0 \end{cases}$$



2) DECREASE BY CONSTANT FACTOR:

$$a^n = \begin{cases} (a^{n/2})^2 & \text{if } n \text{ is even and positive} \\ (a^{(n-1)/2})^2 & \text{if } n \text{ is odd} \\ 1 & \text{if } n = 0 \end{cases}$$



3) VARIABLE SIZE DECREASE:

$$\gcd(m, n) = \gcd(n, m \bmod n)$$

* INSERTION SORT: $(-A[0 \dots n-1])$

for $i \leftarrow 1$ to $n-1$ do

$v \leftarrow A[i]$

$j \leftarrow i-1$

 while $j \geq 0$ and $A[j] > v$ do

$A[j+1] \leftarrow A[j]$

$j \leftarrow j-1$

$A[j+1] \leftarrow v$

Basic Operation.

Ex:

89 | 45 68 90 29 34 17
 45 89 | 68 90 29 34 17
 45 68 89 | 90 29 34 17
 45 68 89 90 | 29 34 17
 29 45 88 89 90 | 34 17
 29 34 5 45 68 89 90 | 17
 17 29 34 45 68 89 90

$$C(u) = \sum_{i=1}^{n+1} \sum_{j=0}^{i-1} 1 = \sum_{i=1}^{n+1} [i-1+1] = \sum_{i=1}^{n+1} i$$

$$= \sum_{i=1}^{n+1} i$$

$$\therefore \sum_{i=1}^n i = \frac{n(n+1)}{2}$$

(*) $C_{\text{worst}} = \frac{(n-1)n}{2} \in \Theta(n^2)$

(*) $C_{\text{best}} = \sum_{i=1}^{n-1} 1 = n-1 \in \Theta(n)$

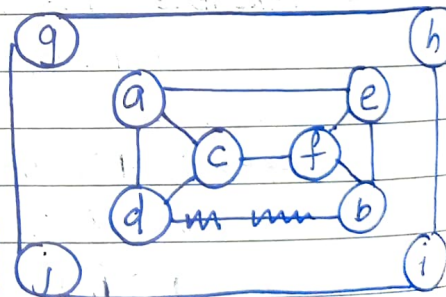
(*) $C_{\text{avg}} = n^2/4 \in \Theta(n^2)$

* DEPTH FIRST SEARCH (DFS):

- It starts a graph traversal at an arbitrary vertex by marking it as visited
- On each iteration, the algo proceeds to unvisited vertex i.e. adjacent to one it is currently in.

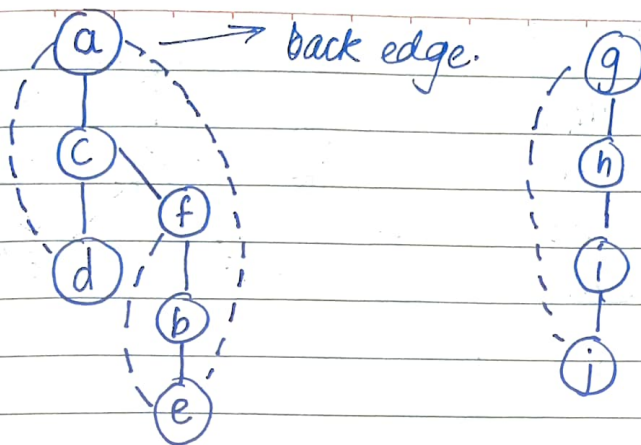
Ex: 1

	$e_{6,2}$				
	b_5	$b_{5,3}$			
$d_{3,1}$	f_4	f_4	$f_{4,4}$		
c_2	c_2	c_2	c_2	$c_{2,5}$	
a_1	a_1	a_1	a_1	a_1	$a_{1,6}$



$j_{10,7}$			
i_9	$i_{9,8}$		
h_8	h_8	$h_{8,9}$	
g_7	g_7	g_7	$g_{7,10}$

$\Rightarrow a c d f b e$
 $\Rightarrow g h i j$



ALGORITHM : DFS (G)

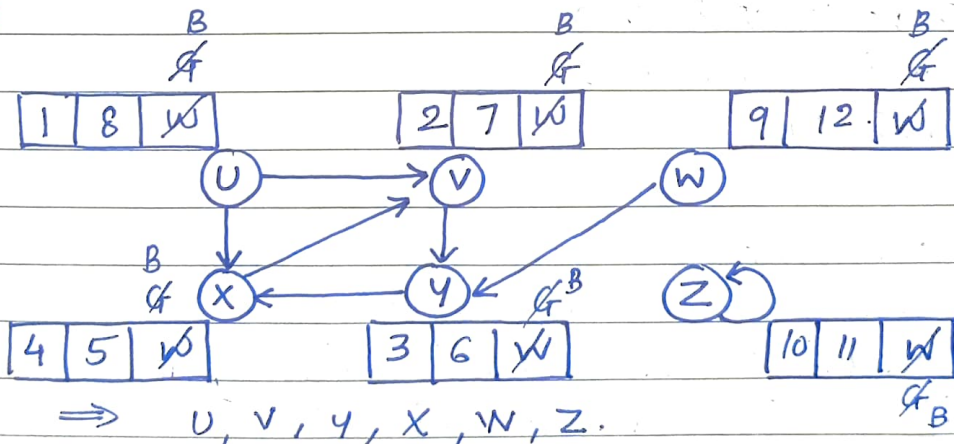
count $\leftarrow 0$
 for each vertex v in V do
 if v is marked with 0
 dfs(v)

dfs(v) :

count $\leftarrow$ count + 1 ; mark v with count
 for each vertex w in V adjacent to v do
 if w is marked with 0
 dfs(w)

METHOD 2 :

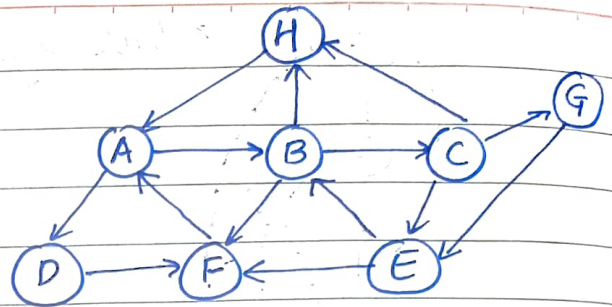
EX2:



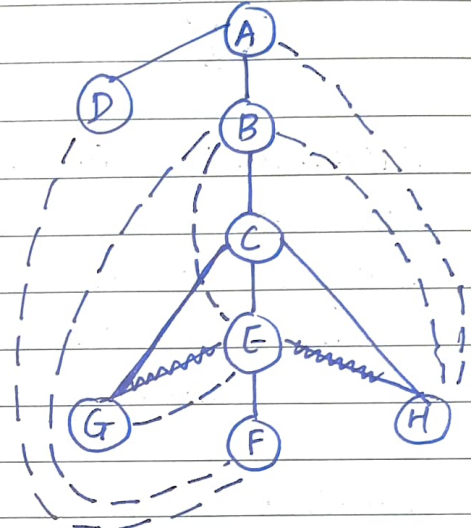
DFS - stack

BFS - queue

$F_{5,1}$			
E_4	$E_{4,2}$	$G_{6,3}$	$H_{7,4}$
C_3	C_3	C_3	C_3
B_2	B_2	B_2	B_2
A_1	A_1	A_1	A_1



$C_{3,5}$			
B_2	$B_{2,6}$	$D_{8,7}$	
A_1	A_1	A_1	$A_{1,8}$



* BREADTH FIRST SEARCH (BFS) :

- concentric manner visiting all the vertex.

ALGORITHM: $BFS(G)$.

count $\leftarrow 0$

for each vertex v in V do

if v is marked with 0
bfs(v)

bfs(v)

count $\leftarrow$ count + 1 ;

while the queue is not empty do

for each vertex w in V adjacent to front vertex do
if w is marked with 0

count $\leftarrow$ count + 1 ;

add w to the queue

remove the front vertex from the queue.

front = 0 (f)
 rear = -1 (r).

- remove front
- insert rear

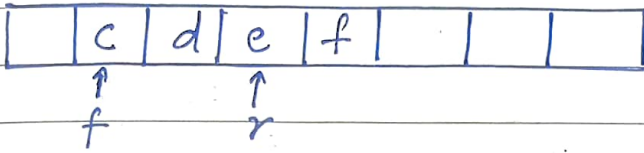
DATE ____/____/____
 Mo Tu We Th Fr Sa Su

Ex 1:

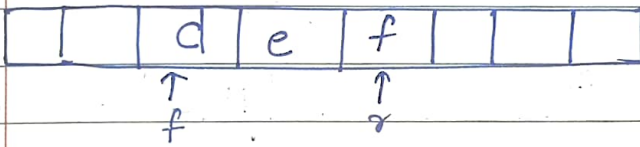


$$\text{adj}(a) = \{c, d, e\}$$

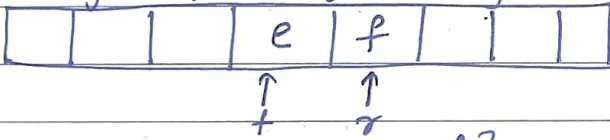
a₁
 c₂
 d₃
 e₄



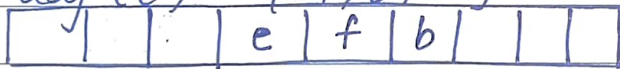
$$\text{adj}(c) = \{a, d, f\}$$



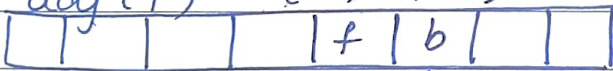
$$\text{adj}(d) = \{a, c\}$$



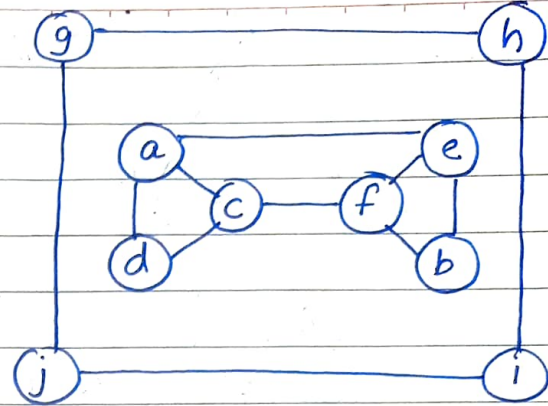
$$\text{adj}(e) = \{a, b, f\}$$



$$\text{adj}(f) = \{b, c, e\}$$

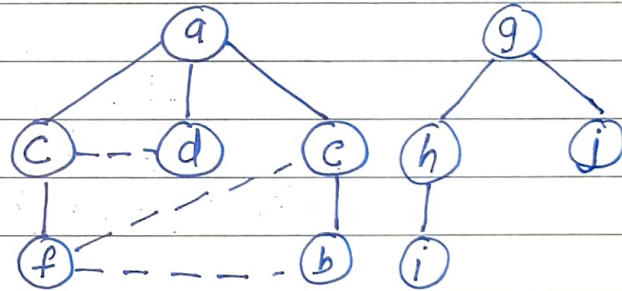


remove(f and b)

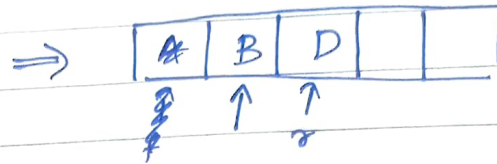
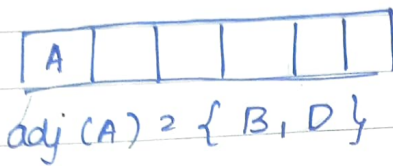
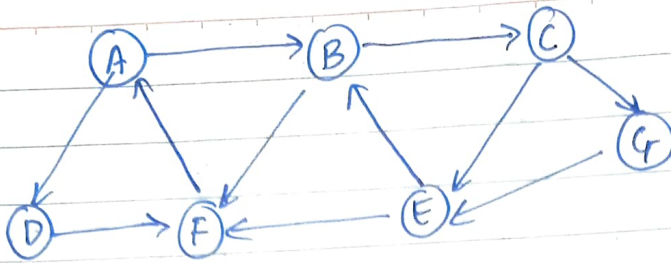


f₅

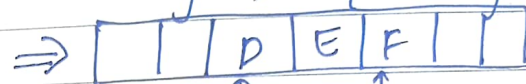
a₁ c₂ d₃ e₄ f₅ b₆
 g₇ h₈ j₉ i₁₀



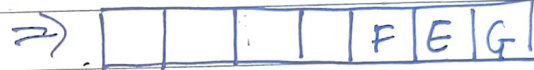
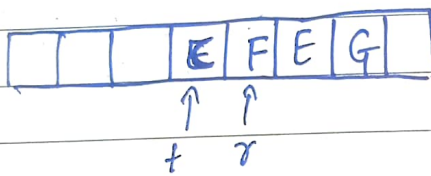
EX2:



adj(B) = {F, C}

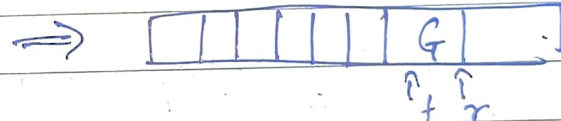
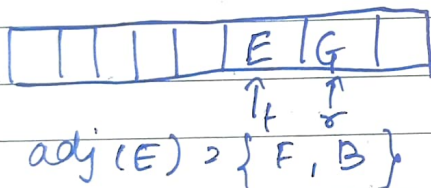


adj(D) = {F}



adj(E) = {B, G}

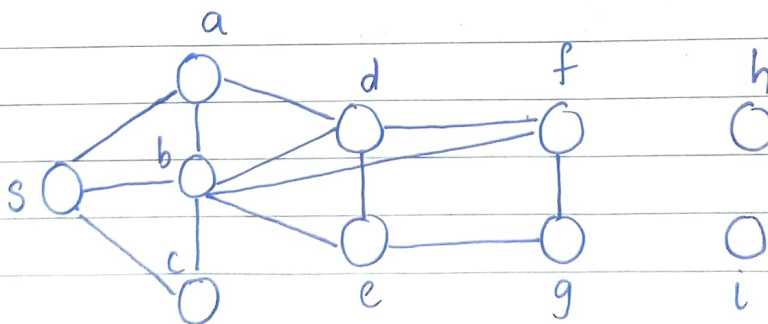
adj(F) = {A}



adj(E) = {F, B}

adj(G) = {E}

EX3:

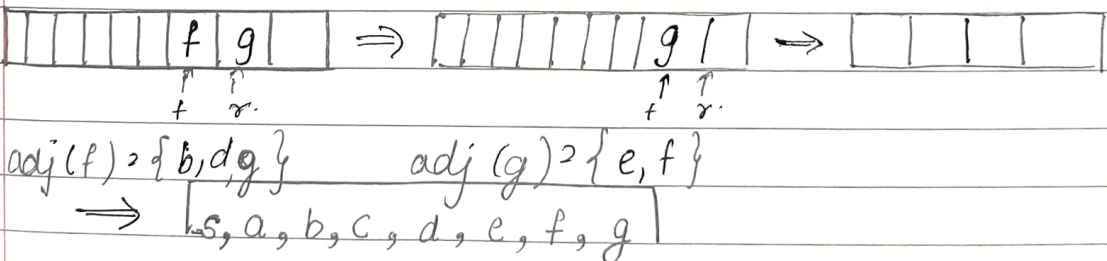
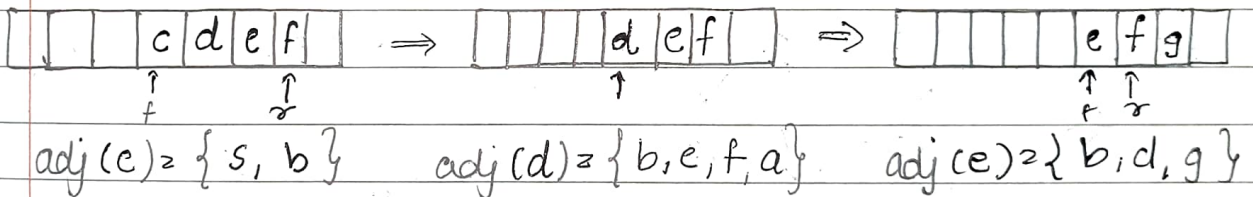
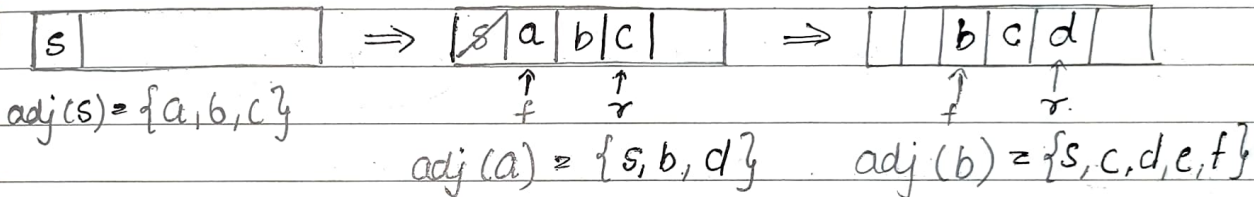


Ans: {s, a, b, c, d, e, f, g}

DIFFERENCE B/W DFS & BFS:

	DFS	BFS
1. Data Structure	a stack	a queue
2. No. of vertex ordering	two ordering	one ordering
3. Edge types (undirected graphs)	tree and back edges	tree and cross edges
4. Applications	connectivity, acyclicity articulation points	acyclicity, minimum edge paths.
5. Efficiency for adjacency matrix	$\theta(V^2)$	$\theta(V^2)$
6. Efficiency for adjacency list	$\theta(V + E)$	$\theta(V + E)$

Ex 3:



DAG - Directed Acyclic Graph

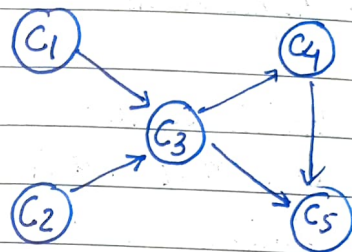
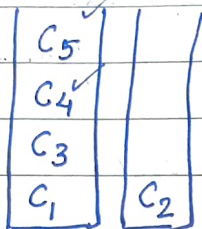
- * Incoming edge (Indegree) = 0

* TOPOLOGICAL SORTING:

- It is not possible to have solution if a digraph has a directed cycle.
- If the graph is DAG, then we obtain the solution for the problem.

METHOD-1 : dfs based :

Ex :

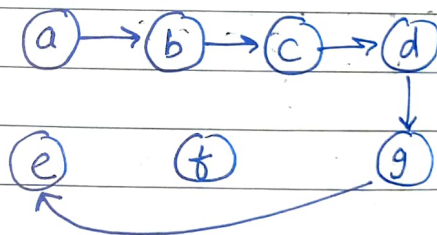
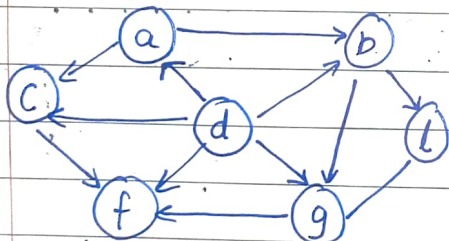


Popping off order : C₅, C₄, C₃, C₁, C₂

Soln: Topologically sorted list : C₂ → C₁ → C₃ → C₄ → C₅

METHOD-2 : decrease by one and conquer :

Ex :



TRANSFORM AND CONQUER.

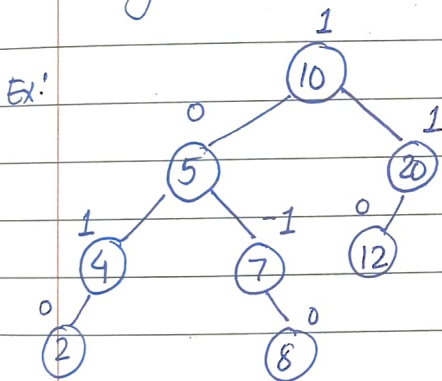
- It is a 2 stage procedure method:
- (i) Transformation stage, problem instance is modified
- (ii) Conquering stage, the problem is solved.

3 major variations of transformation:

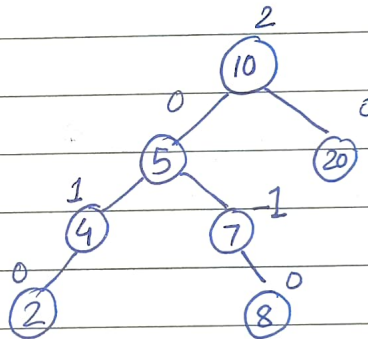
- simplication.
- representatⁿ change.
- problem reduction.

* BRANCHED SEARCH TREE: (i) AVL

- AVL tree: is a binary search tree in which balance factor of every node, which is defined as the diff btw height of node's left and right subtree is either 0, -1 or +1.



(i) AVL Tree

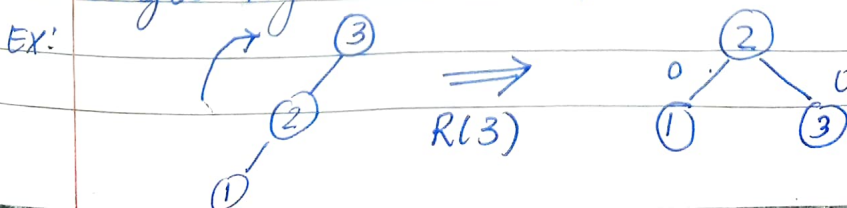


(ii) BST but not AVL tree.

- Balance factor of 2 and -2, we do rotation to make it an AVL tree.

- * TYPES OF ROTATION:
- single-right rotation.
 - single-left rotation.
 - double left followed by right
 - double right followed by left.

- Single right rotation / R-rotation:



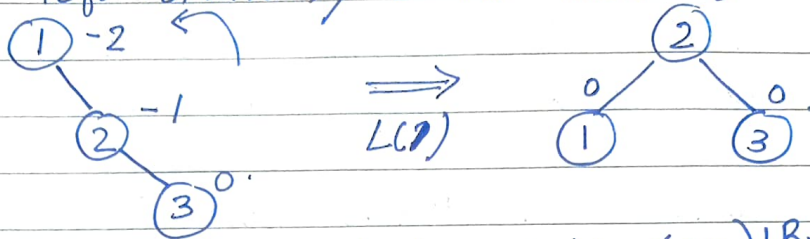
NOTE: (i) Nearest balance factor is considered from the leaf node to transform.

(ii) Start newly inserted node.

DATE / /
Mo Tu We Th Fr Sa Su

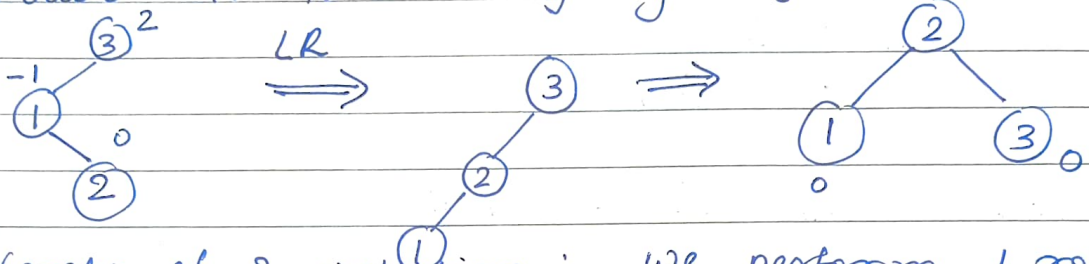
(i) Single left rotation / L-rotation :

EX:



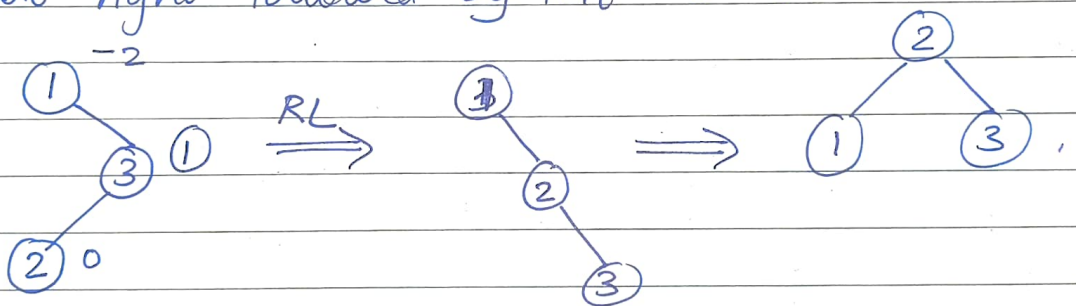
(ii) Double left followed by right. $(+2)LR$.

EX:

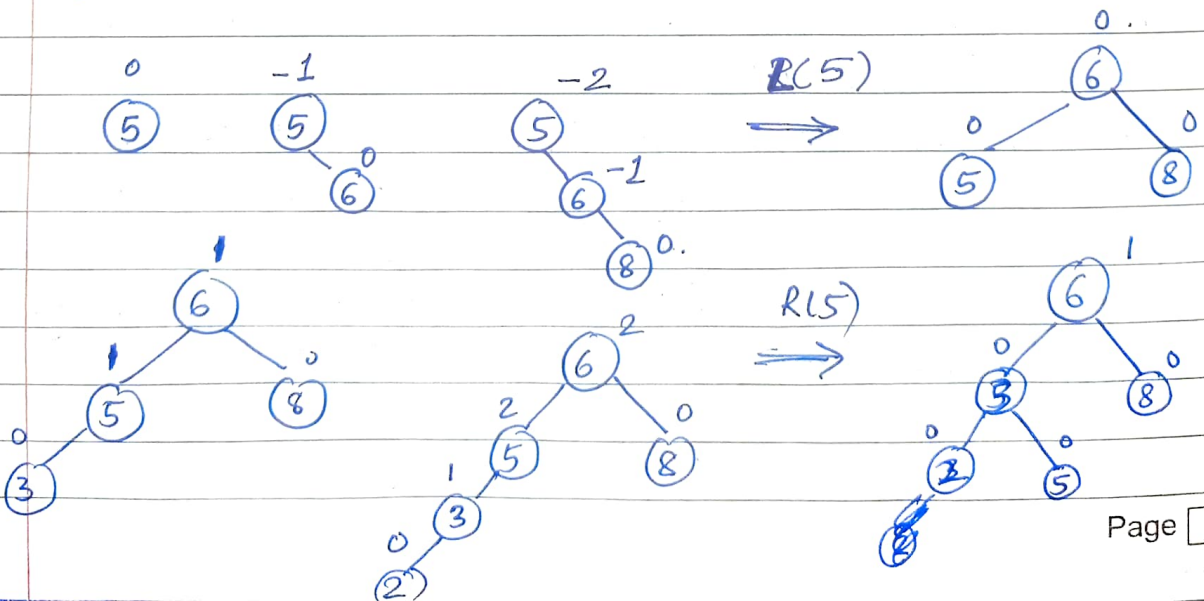


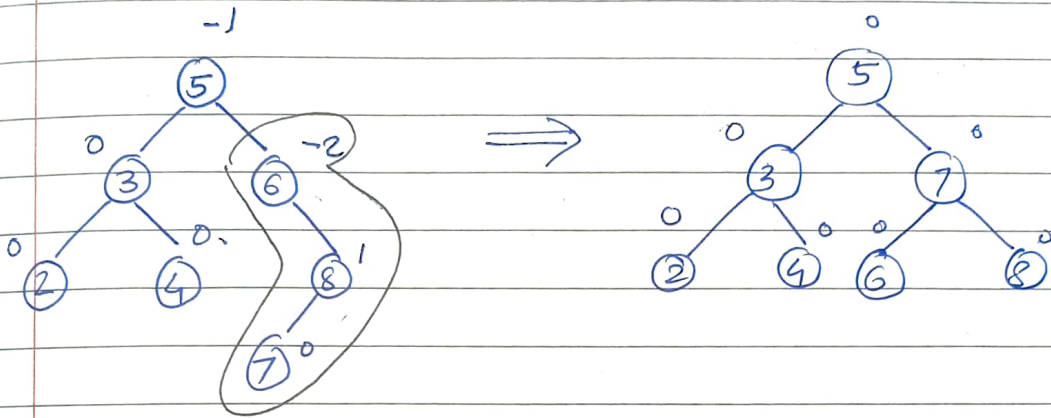
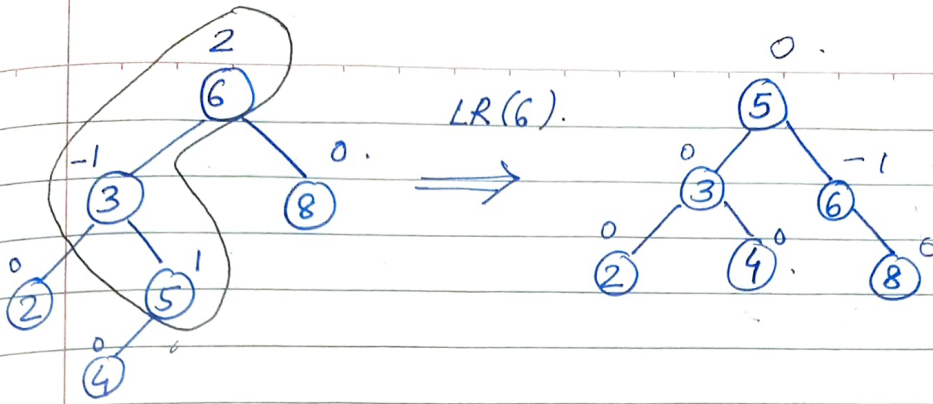
(iii) Combo of 2 rotation : we perform L rotation of the subtree and then R-rotation on the obtained tree.

(iv) Double right followed by left : $(-2)RL$



Q. Construction of an AVL tree for the list 5, 6, 8, 3, 2, 4, 7 by successive insertion.





(c) Efficiency of AVL trees :

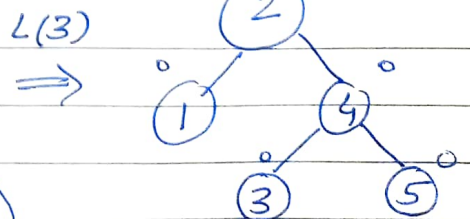
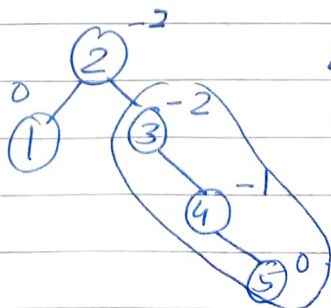
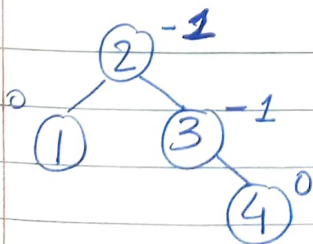
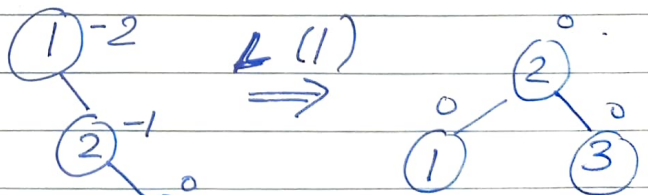
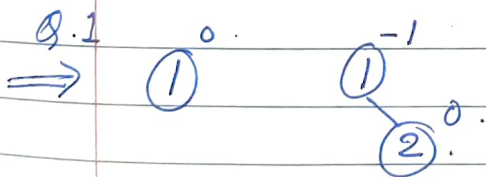
$$\lfloor \log_2 n \rfloor \leq h < 1.4405 \log_2 (n+2) - 1.3277$$

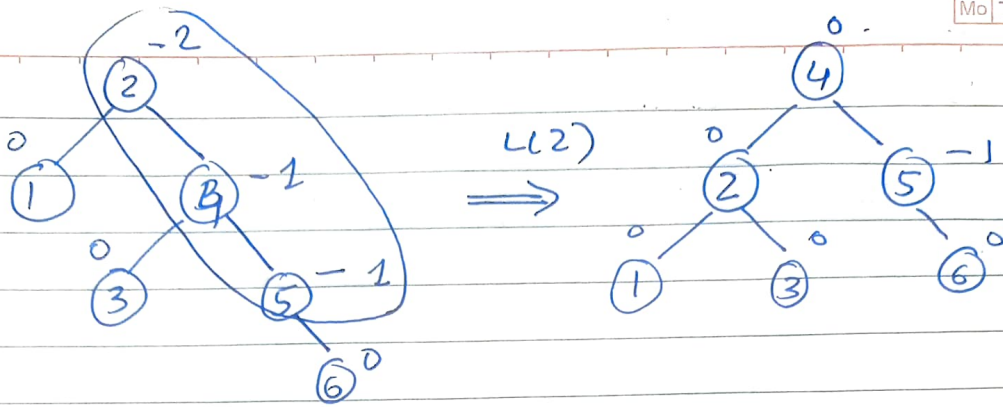
h - height

n - nodes

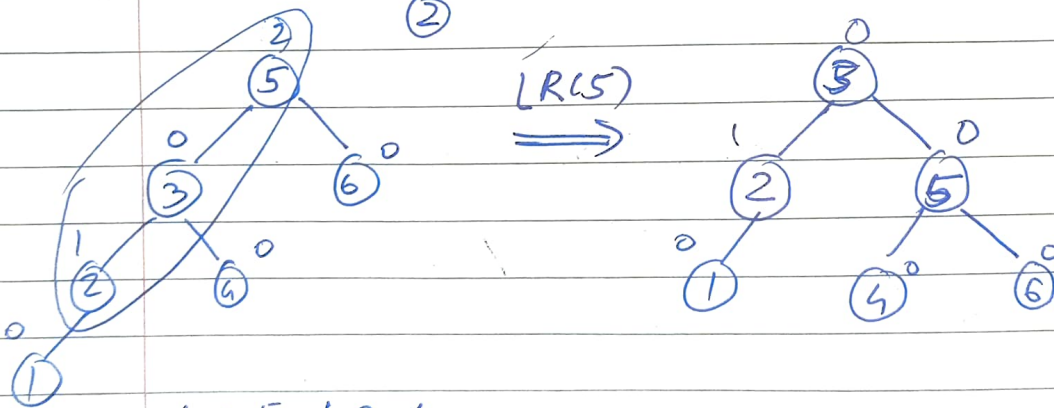
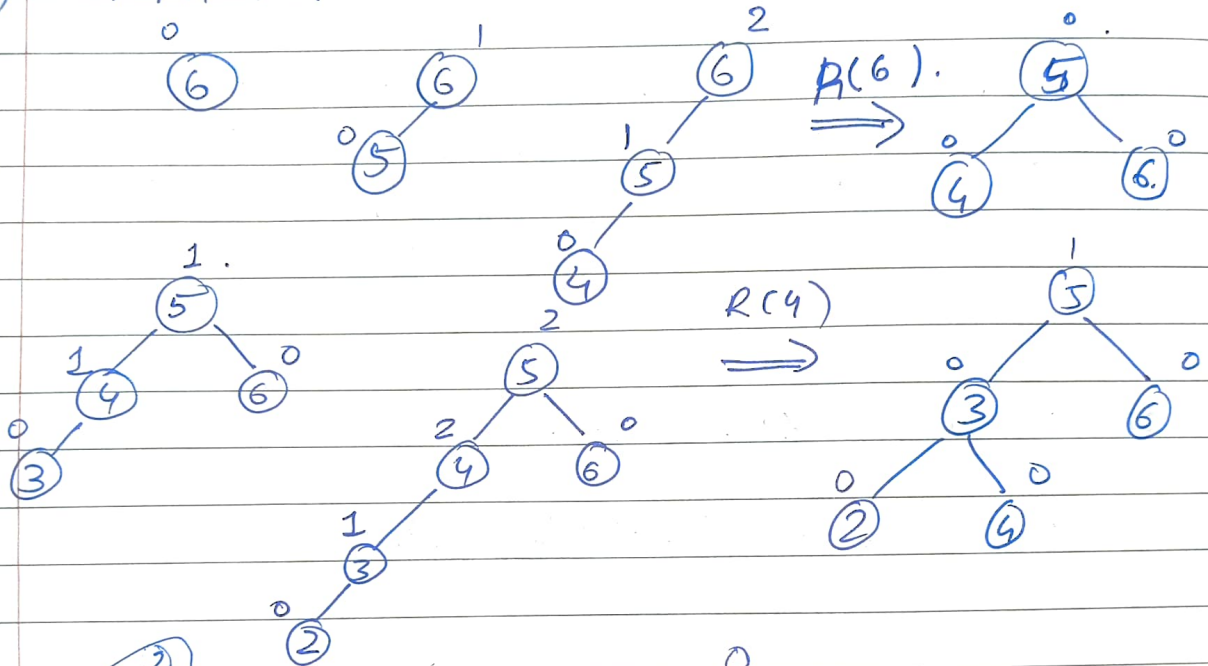
Q.) 1, 2, 3, 4, 5, 6.

Q.) 6, 5, 4, 3, 2, 1 Q.)

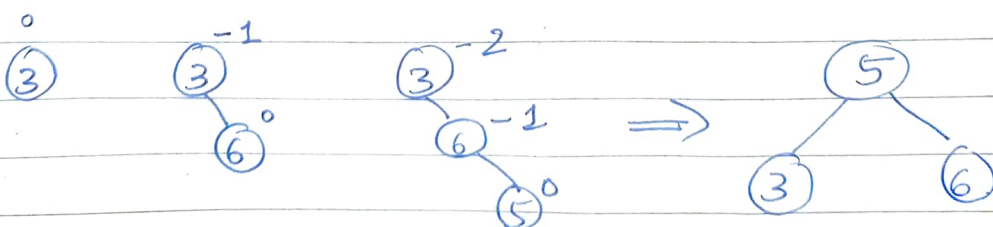




Q.2.) 6, 5, 4, 3, 2, 1.



Q.3 3, 6, 5, 1, 2, 4.



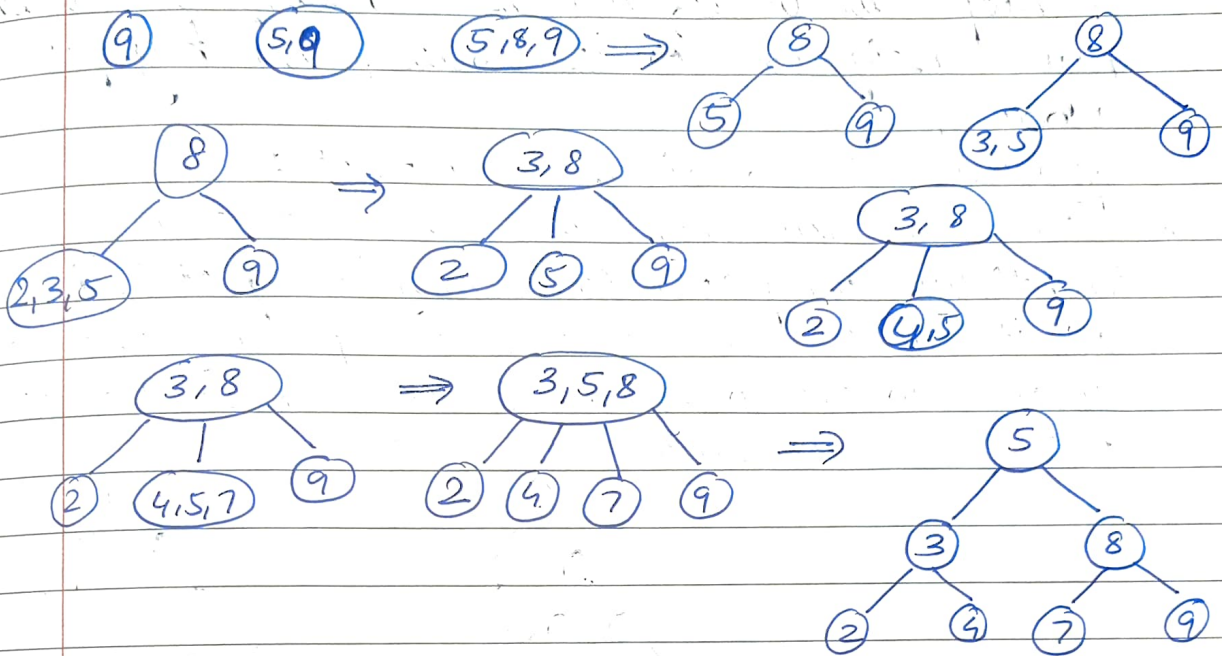
2-nodus, 3-podster
1-value, 2-podster

DATE / /

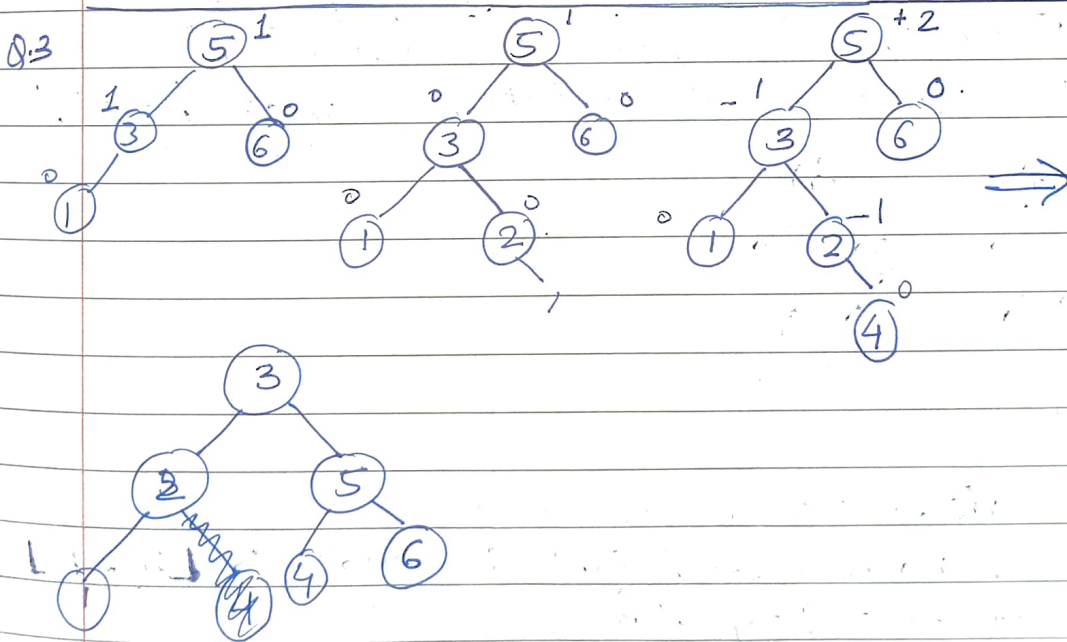
Mo Tu We Th Fr Sa Su

* 2-3 TREES :

Q. Construct a 2-3 tree for the list 9, 5, 8, 3, 2, 4, 7



$$O(\log_2 N)$$



best DS $\Rightarrow$ priority queue $\rightarrow$ Heap. (★)

DATE 1 / 3 / 25

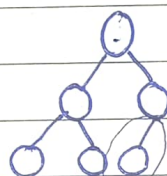
Mo Tu We Th Fr Sa Su

* HEAP AND HEAPSORT:

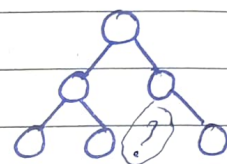
- A heap can be defined as a binary tree with keys assigned to its nodes, one key per node,

provided 2 conditions are met:

- 1) The shape property - the binary tree is essentially complete. (or simply complete), i.e. all its levels are full except possibly the last level, where only some rightmost leaves may be missing.

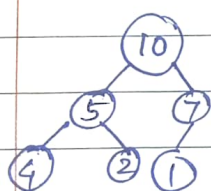


✓ almost complete.

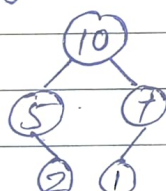


X almost complete

- 2) The parental dominance or heap property - the key in each node is greater than or equal to the keys in its children. (max heap).
default



Heap ✓



X

* Properties:

1. There exists exactly 1 essentially complete binary tree with n nodes. Its height is equal to $\lfloor \log_2 n \rfloor$.
2. The root of a heap always contains its largest element.
3. A node of a heap considered with all its descendant is also a heap.
4. A heap can be implemented as an array by recording its elements in top-down, left-to-right fashion. It is convenient to store the heap's element in position 1 through n of such an array leaving

(Imp)

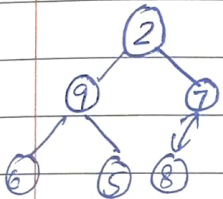
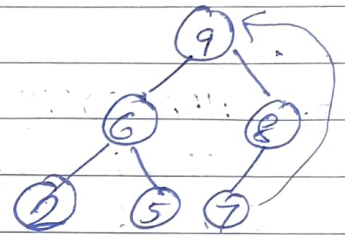
* BOTTOM-UP HEAP CONSTRUCTION:

Heapification: last parental node (subtree).

- Check whether 2 child (compare among them)
- considered the largest node
- exchange with parent node

Ex:

9 6 8 2 5 7
7 6 8 2 5 9



2
9 8
6 5 7

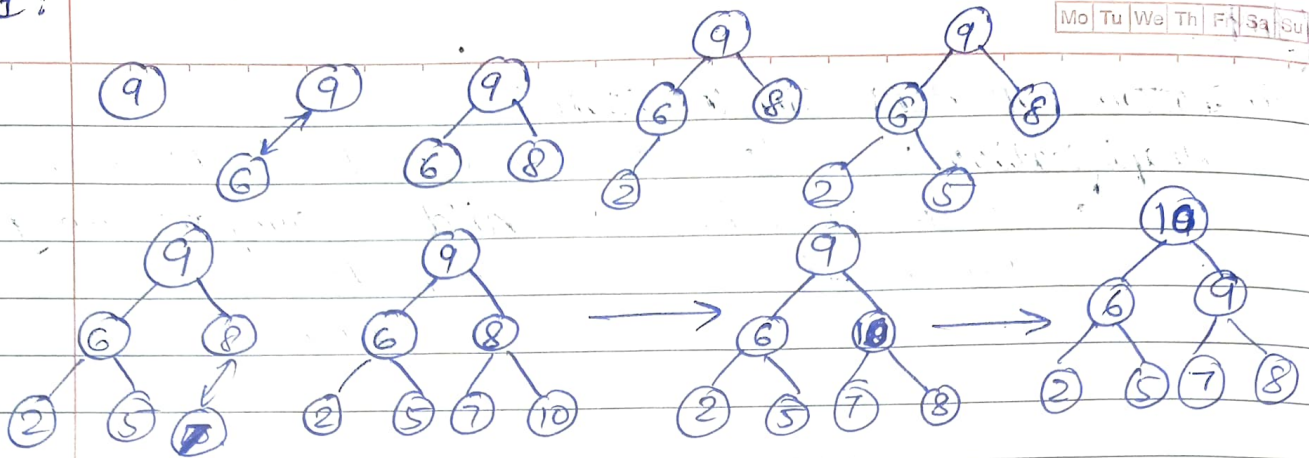
* ALGORITHM HeapBottomUp($H[1..n]$).for $i \leftarrow \lfloor n/2 \rfloor$ down to 1 do $k \leftarrow i; v \leftarrow H[k]$ heap $\leftarrow$ falsewhile not heap and $2 * k \leq n$ do. $j \leftarrow 2 * k$ if $j < n$ if $H[j] < H[j+1]$ $j \leftarrow j+1$ if $v \geq H[j]$ heap $\leftarrow$ trueelse $H[k] \leftarrow H[j]; k \leftarrow j$ $H[k] \leftarrow v.$ * EFFICIENCY : $\Theta(n)$.

* TOP-DOWN HEAP CONSTRUCTION: (Not required)

EX 1:

9, 6, 8, 2, 5, 7, 10

DATE: / /
Mo Tu We Th Fr Sa Su

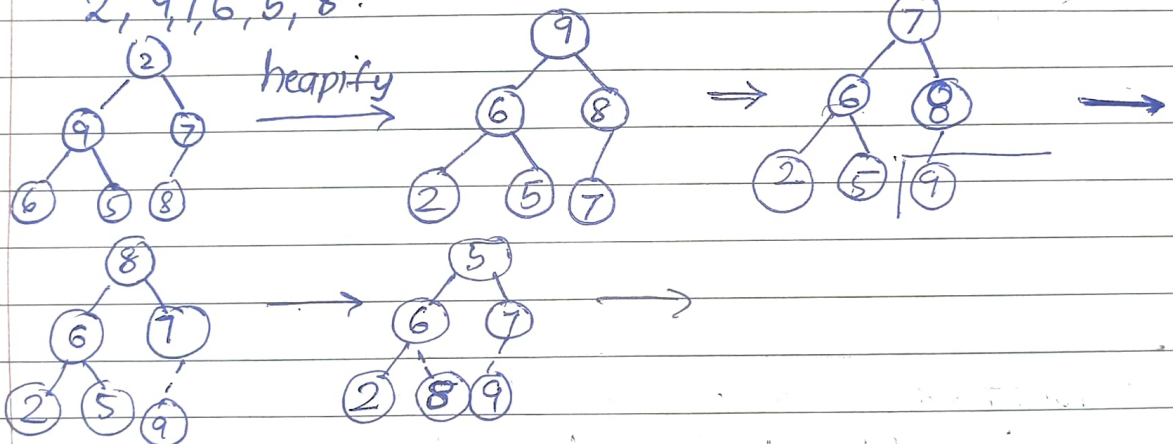


* HEAPSORT:

STEP 1: Construction of heap
STEP 2:

EX:

2, 9, 7, 6, 5, 8



*

Heapsort TIME EFFICIENCY (Worst/ Avg): $\Theta(n \log n)$.

(*) TIME AND SPACE TRADEOFFS (*)

- preprocess the input of the given problem, then that extra info accelerates the execution, this process is called input enhancement.
- Prestructuring - adding extra info to the input.

* SORTING BY COUNTING:

	0	1	2	3	4	5	
$A[0 \dots 5]$	62	31	84	96	19	47	$\text{if } (A[i] < A[j])$
	i	j					$\text{count}[j]++$
	0	0	0	0	0	0	$\text{else count}[j]++$
	3	0	1	1	0	0	
	3	1	2	2	0	1	
	3	1	4	3	0	1	
	3	1	4	5	0	1	
	3	1	4	5	0	2	$\Rightarrow$ Input Enhancement
$S[0 \dots 5]$	19	31	47	62	84	96	

* ALGORITHM: ComparisonCountingSort ($A[0 \dots n-1]$).for $i \leftarrow 0$ to $n-1$ do $\text{count}[i] \leftarrow 0$ for $i \leftarrow 0$ to $n-2$ dofor $j \leftarrow i+1$ to $n-1$ doif $A[i] < A[j]$ $\text{count}[j] \leftarrow \text{count}[j] + 1$ else $\text{count}[i] \leftarrow \text{count}[i] + 1$ for $i \leftarrow 0$ to $n-1$ do $S[\text{count}[i]] \leftarrow A[i]$ return S .* DISTRIBUTION COUNTING:

EX:

Array $\Rightarrow$

13	11	12	13	12	12
----	----	----	----	----	----

Set - $\{11, 12, 13\}$

Array values	11	12	13
Frequencies	1	3	2

Distribution values	1	4	6
---------------------	---	---	---

Array $\Rightarrow$

11	12	13
0	0	0

 D. $l = 11$
 Freq $\Rightarrow$ $u = 13$.

DATE / /
 Mo Tu We Th Fr Sa Su

ALGORITHM:

for $j \leftarrow 0$ to $u-1$ do $D[j] \leftarrow 0$ // initialize freq
 for $i \leftarrow 0$ to $n-1$ do $D[A[i]-1] \leftarrow D[A[i]-1] + 1$ // compute freq
 for $j \leftarrow 1$ to $u-1$ do $D[j] \leftarrow D[j-1] + D[j]$ // reuse distribution
 for $i \leftarrow n-1$ down to 0 do
 $j \leftarrow A[i]-1$
 $S[D[j]-1] \leftarrow A[i]$
 $D[j] \leftarrow D[j]-1$
 return S.

Ex: A

0	1	2	3	4	5
13	11	12	13	12	12

 S

11	12	12	12	13	13
----	----	----	----	----	----

 D

1	3	2
---	---	---

* HORSPPOOL'S ALGORITHM:

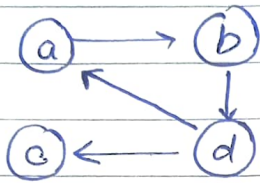
DYNAMIC PROGRAMMING.

DATE 18 / 3 / 25
Mo Tu We Th Fr Sa Su

* WARSHALL'S ALGORITHM:
 (•) $x \text{ to } y (\text{path}) = 1$
 (•) $x \text{ to } y (\text{no path}) = 0.$

Ex:
 $\Theta(n^3)$

Transitive Closure: (T)



$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

$$T = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$T[x, y] = 1$$

$$T[\underbrace{x \text{ to } k}_0 \text{ and } \underbrace{x \text{ to } y}_1] = 1$$

* ALGORITHM: WARSHALL ($A[1 \dots n, 1 \dots n]$)

$$R^{(0)} \leftarrow A$$

for $k \leftarrow 1$ to n do

for $i \leftarrow 1$ to n do

for $j \leftarrow 1$ to n do

$$R^{(k)}[i, j] \leftarrow R^{(k-1)}[i, j] \text{ or } R^{(k-1)}[i, k] \text{ and } R^{(k-1)}[k, j]$$

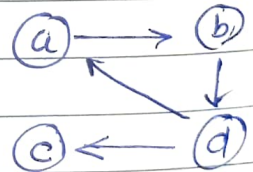
return $R^{(n)}$

(•) Efficiency: $\Theta(n^3)$.

Ex 2: $R^{(0)} = A$ = Adjacency matrix.

$$R^{(0)} = \begin{bmatrix} a & b & c & d \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

$$R^{(1)} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$



(*) initialize with ∞ (9999) - adj matrix and update

DATE ____/____/____

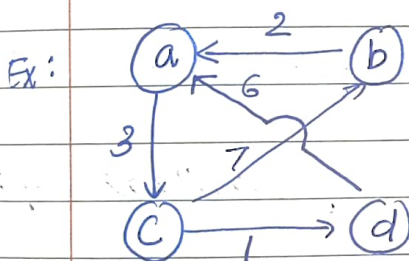
Mo Tu We Th Fr Sa Su

$$R^{(2)} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$R^{(3)} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$R^{(4)} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

* FLOYD'S ALGORITHM : (for all pair shortest path).



$$W = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{matrix}$$

$$D = \begin{bmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 7 & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{bmatrix}$$

* ALGORITHM : FLOYD ($W[1 \dots n, 1 \dots n]$)

$D \leftarrow W$

for $k \leftarrow$

Via

Efficiency : $\theta(n^3)$.

adj matrix

DATE / /

Mo Tu We Th Fr Sa Su

$$D^{(0)} = \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix}$$

$$D^{(1)} = \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{bmatrix}$$

$$D^{(2)} = \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ 9 & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{bmatrix}$$

$$D^{(3)} = \begin{bmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 9 & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{bmatrix}$$

$$D^{(4)} = \begin{bmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 7 & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{bmatrix}$$

6/8m

* KNAPSACK PROBLEM USING DYNAMIC PROGRAMMING :

W - Total available capacity

$v_1, v_2 \dots v_n$ - value of items

$w_1, w_2 \dots w_n$ - weight of items

j - available capacity

$$c) \quad v[i, j] = \begin{cases} \max \{v[i-1, j], v_i + v[i-1, j-w_i]\} & \text{if } j-w_i \geq 0 \\ v[i-1, j] & \text{if } j-w_i < 0. \end{cases}$$

Ex: 1

items	weight	value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

Capacity $W = 5$.

items - row.

avail capc - value - column.

DATE / /

Mo Tu We Th Fr Sa Su

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	10	20	20	320
4	0	10	15	25	30	37

(i)

← available capacity →

y).

$$V[1,1] = M(0, 1) \rightarrow V[0,0]$$

$$V[1,2] = M(0, 12+0) = 12$$

$$V[1,3] = M(0, 12+0) \rightarrow V[0,1] = 12$$

$$V[1,4] = M(0, 12+0) \rightarrow V[0,2] = 12$$

$$V[1,5] = M(0, 12+0) \rightarrow V[0,3] = 12$$

$$V[2,1] = M(0, 10+0) \rightarrow V[0,0] = 10$$

$$V[2,2] = M(12, 10+0) \rightarrow V[1,1] = 12$$

$$V[2,3] = M(12, 10+12) \rightarrow V[1,2] = 22$$

$$V[2,4] = M(12, 10+12) \rightarrow V[1,3] = 22$$

$$V[2,5] = M(12, 10+12) \rightarrow V[1,4] = 22$$

$$V[3,1] = M(10, 20+0) \rightarrow V[2,2] = 30$$

$$V[3,2] = M(12, 20+0) \rightarrow V[2,0] = 12$$

$$V[3,3] = M(22, 20+0) \rightarrow V[2,1] = 22$$

$$V[3,4] = M(22, 20+10) \rightarrow V[2,2] = 30$$

$$V[3,5] = M(22, 20+12) \rightarrow V[2,3] = 32$$

$$V[4,1] = M(10, 15+0) \rightarrow V[3,0] = 10$$

$$V[4,2] = M(10, 15+0) \rightarrow V[3,1] = 15$$

$$V[4,3] = M(22, 15+10) \rightarrow V[3,2] = 25$$

$$V[4,4] = M(30, 15+10) \rightarrow V[3,3] = 30$$

$$V[4,5] = M(32, 15+22) \rightarrow V[3,4] = 37$$